

Министерство науки и высшего образования Российской Федерации
«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по производственной практике

**Тема 2: «Система анализа криптовалютного рынка на основе открытых
данных бирж»**

Выполнил: студент Khong Dai Nam

Группа: M3310

Руководитель: Повышев В. В.

Санкт-Петербург, 2026

1. Постановка задачи

Требовалось разработать систему, которая самостоятельно собирает рыночные данные криптовалют из открытых источников, сохраняет их в базе данных и выполняет количественный анализ. С точки зрения практиканта задача состоит в построении воспроизводимого конвейера обработки данных - от обращения к внешнему API до готового аналитического отчёта. Прикладная цель - оценить риск (волатильность) основных криптоактивов и тесноту связи их движений: при сильной корреляции диверсификация между активами слабо снижает риск.

Задача декомпозирована на подзадачи: спроектировать архитектуру, модель данных и процессы обработки; реализовать загрузку котировок BTC и ETH из API CoinGecko в реляционную СУБД с защитой от дублирования; вычислить базовые статистики, волатильность и коэффициент корреляции Пирсона; визуализировать динамику цен и сформировать PDF-отчёт; обеспечить локальный запуск и демонстрацию через Swagger. Целевая платформа - .NET 10 (C#), без контейнеризации.

2. Проектирование программного продукта

2.1. Архитектура и компоненты

Система построена по слоистой архитектуре и логически разделена на шесть модулей (рисунок 1). Физически код сгруппирован в два проекта .NET: веб-приложение CryptoAnalysis.Api (HTTP-вход и Swagger) и библиотеку CryptoAnalysis.Core (приём, хранение, анализ, визуализация, отчёты). Поток данных однонаправленный: CoinGecko → Ingestion → Data (PostgreSQL) → Analytics → Charts/Reports → PDF.

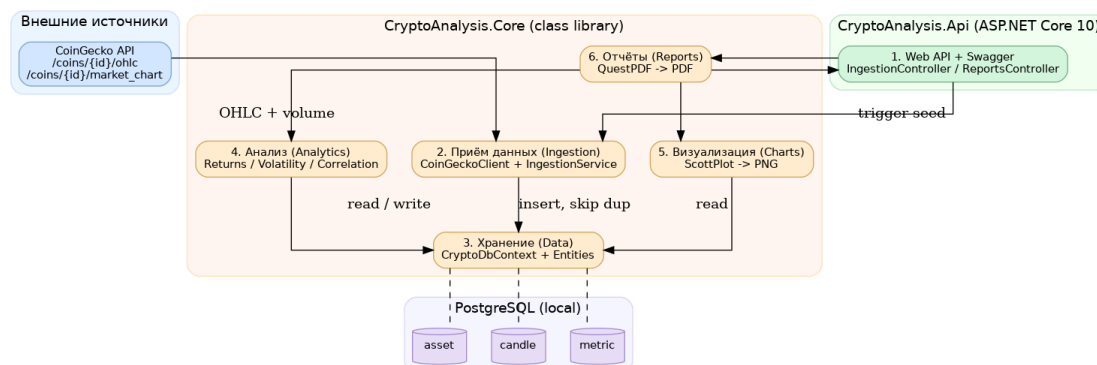


Рисунок 1 - Архитектура системы: шесть логических модулей и поток данных

Таблица 1 - Логические модули системы

№	Модуль	Назначение и состав
1	Web API + Swagger	HTTP-вход, документация API, запуск ETL и отчёта (Ingestion/Reports/CandlesController)
2	Ingestion	Вызов CoinGecko, разбор JSON, загрузка с отсечением дубликатов (CoinGeckoClient, IngestionService)
3	Data	Хранение через EF Core (CryptoDbContext, сущности Asset/Candle/Metric)
4	Analytics	Доходности, волатильность, корреляция Пирсона (Returns, AnalysisService, MathNet)
5	Charts	Линейные графики цен → PNG (PriceChartService, ScottPlot)
6	Reports	Сборка PDF с таблицами и графиками (PdfReportService, QuestPDF)

2.2. Выбор технологий

Стек выбран исходя из требований локального запуска и кроссплатформенности (macOS Apple Silicon): .NET 10 / C# (LTS, нативный arm64), ASP.NET Core 10 + Swagger (REST и автодокументация), PostgreSQL + EF Core 10 с Npgsql (миграции, типобезопасные запросы), MathNet.Numerics (статистика и корреляция), ScottPlot 5 (графики), QuestPDF (генерация PDF), xUnit (тесты).

2.3. Интерфейсы

Внешний интерфейс - REST API, документируемое через Swagger (таблица 2).

Таблица 2 - Программный интерфейс (REST API)

Метод	Маршрут	Назначение
POST	/ingest	Загрузка OHLCV BTC/ETH в БД (с отсечением дубликатов)
GET	/assets, /candles	Список активов; просмотр сырых данных OHLCV
POST	/reports	Анализ и формирование PDF-отчёта

2.4. UML-диаграмма и взаимодействие компонентов

Обработку запроса POST /reports иллюстрирует диаграмма последовательности (рисунок 2): контроллер делегирует работу сервису отчётов, который запускает анализ (чтение свечей, расчёт и сохранение показателей), построение графиков и сборку PDF. Зависимости направлены от Api к Core и далее к инфраструктуре, что реализуется встроенным DI-контейнером ASP.NET Core.

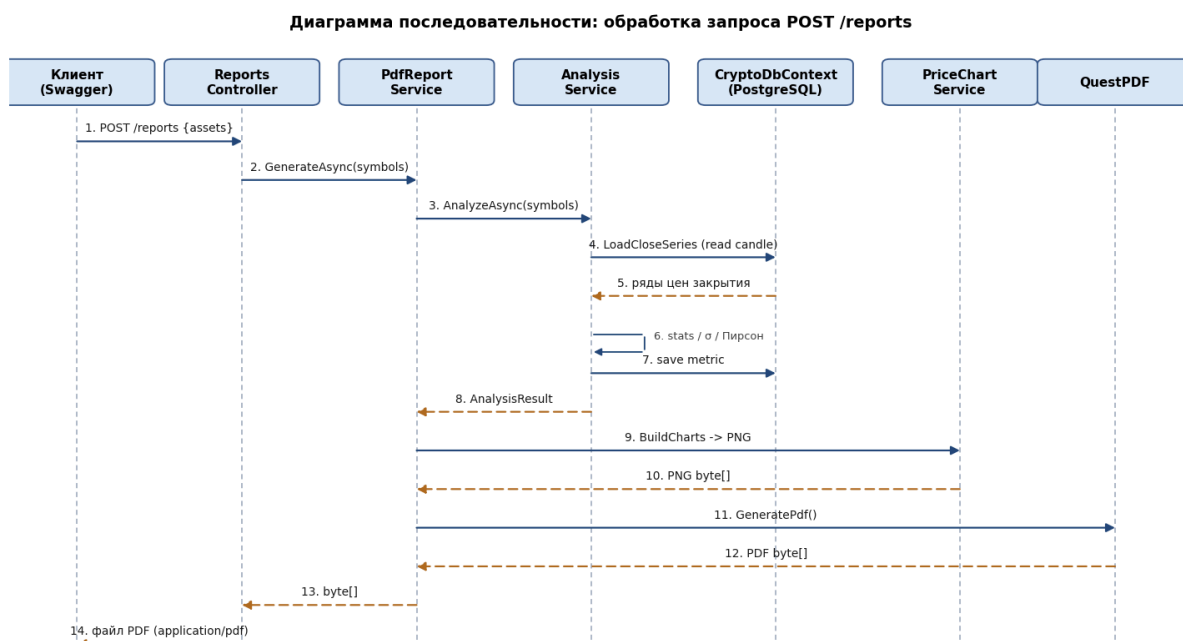


Рисунок 2 - Диаграмма последовательности UML: обработка запроса POST /reports

3. Проектирование базы данных

На концептуальном уровне выделены три сущности: актив (Asset), свеча OHLCV (Candle) и показатель анализа (Metric); базовая связь - «один ко многим» от актива к свечам и показателям. ER-модель приведена на рисунке 3.

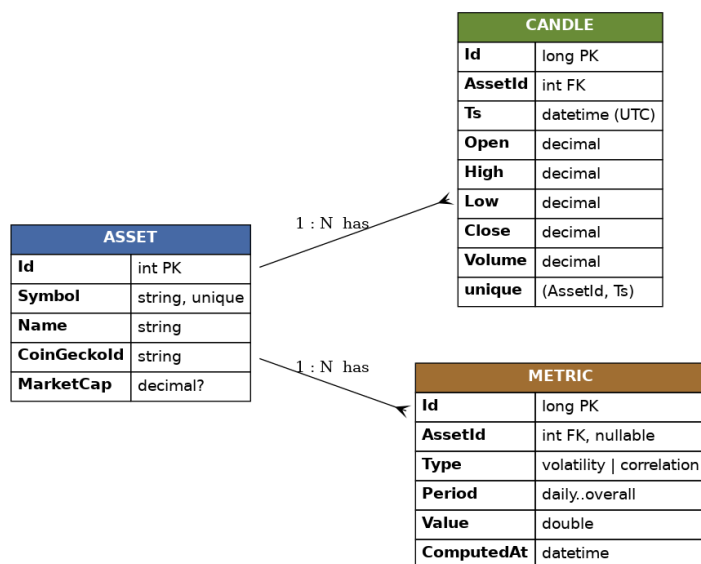


Рисунок 3 - ER-модель: таблицы asset, candle, metric

Логическая модель: asset (Id PK, Symbol, Name, CoinGeckoId, MarketCap) - справочник активов; candle (Id PK, AssetId FK, Ts, Open/High/Low/Close/Volume) - свечи OHLCV; metric (Id PK, AssetId FK nullable, Type, Period, Value, ComputedAt) - результаты анализа (AssetId = NULL для показателей парного уровня, например корреляции BTC–ETH).

Ограничения целостности: первичные ключи во всех таблицах; внешние ключи `candle.AssetId` и `metric.AssetId` → `asset.Id` (ссылочная целостность); уникальное ограничение (`AssetId`, `Ts`) в `candle` предотвращает дублирование свечей; индекс по (`AssetId`, `Ts`) ускоряет выборку временных рядов; уникальность `asset.Symbol` исключает дублирование активов.

4. Проектирование процессов обработки данных

Источник данных - публичный API CoinGecko (бесплатный demo-доступ). Загрузка построена по схеме ETL. Extract: `CoinGeckoClient` через типизированный `HttpClient` получает OHLC (`GET /coins/{id}/ohlcv`) и дневные объёмы (`GET /coins/{id}/market_chart`); метки времени переводятся из Unix-времени в UTC. Transform: элементы JSON отображаются в структуру `RawCandle`, объёмы сопоставляются со свечами по календарной дате (усечение до суток UTC), записи преобразуются в сущность `Candle`. Load: `IngestionService` добавляет только новые свечи, отсекая дубликаты по (`AssetId`, `Ts`), что делает повторную загрузку идиempотентной.

Аналитическая обработка: для каждого актива из базы читается ряд цен закрытия, вычисляются доходности, базовые статистики и волатильность; для пары активов рассчитывается коэффициент Пирсона; результаты сохраняются в таблицу `metric` и используются при формировании отчёта. Важное ограничение источника: demo-тариф возвращает OHLC с шагом 4 дня при `days = 180` (около 45 свечей на актив) - это реальные данные, достаточные для расчётов, что учитывается при интерпретации.

5. Реализация программного продукта

Решение состоит из двух проектов (`Api`, `Core`) и проекта тестов. Ключевые классы: `CoinGeckoClient` (HTTP-клиент), `IngestionService` (загрузка с дедупликацией), `CryptoDbContext` (контекст EF Core), `Returns` (расчёт доходностей), `AnalysisService` (статистика, волатильность, Пирсон), `PriceChartService` (графики `ScottPlot`), `PdfReportService` (сборка PDF в `QuestPDF`). Используемые библиотеки: ASP.NET Core 10 + Swashbuckle 7.2, EF Core 10 + Npgsql 10, MathNet.Numerics 5.0, ScottPlot 5, QuestPDF (Community), xUnit.

Особенности реализации: типизированный `HttpClient` через `IHttpClientFactory`; автоматическое применение миграций при старте

(db.Database.Migrate()); сквозная асинхронность (async/await, CancellationToken); идемпотентная загрузка; чистые функции расчёта доходностей. Пример ключевого фрагмента - расчёт логарифмических доходностей с защитой от неположительных цен:

```
public static double[] Log(IReadOnlyList<double> closes)
{
    if (closes.Count < 2) return Array.Empty<double>();
    var r = new double[closes.Count - 1];
    for (int i = 1; i < closes.Count; i++)
    {
        var prev = closes[i - 1];
        r[i - 1] = (prev <= 0 || closes[i] <= 0) ? 0 : Math.Log(closes[i] / prev);
    }
    return r;
}
```

6. Реализация алгоритмов

Анализ выполняется над рядом цен закрытия, отсортированным по времени. Простая и логарифмическая доходности:

$$r_{\square} = (Close_{\square} - Close_{\square-1}) / Close_{\square-1} \quad r_{\square} = \ln(Close_{\square} / Close_{\square-1})$$

Волатильность - стандартное отклонение логарифмических доходностей; годовая получается умножением на корень из числа свечей в году:

$$\sigma = \sqrt{(1/(n-1)) \cdot \sum (r_{\square} - \bar{r})^2)} \quad \sigma_{год} = \sigma \cdot \sqrt{N}, \quad N = 365/\Delta_{дней}$$

Коэффициент корреляции Пирсона между доходностями двух активов:

$$\rho(X, Y) = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{(\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2)}$$

Особенности и оптимизации: доходности реализованы как чистые функции; среднее/медиана/СКО - средствами MathNet; перед расчётом корреляции ряды BTC и ETH выравниваются по общим меткам времени; для дедупликации используется HashSet (поиск O(1)). Оценка сложности: расчёт доходностей и статистик - O(n), медиана требует сортировки - O(n·log n), корреляция - O(n); итоговая сложность анализа O(n·log n). При n ≈ 45 вычисления выполняются практически мгновенно.

7. Экспериментальная часть

Данные: реальные котировки BTC и ETH в USD за период 22.12.2025 - 16.06.2026, по 45 свечей на актив с шагом ≈ 4 дня (параметры запроса: vs_currency = usd, days = 180). Сценарий эксперимента повторяет полный цикл работы системы: запуск приложения (dotnet run, автоприменение миграций) → POST /ingest (загрузка котировок) → POST /reports (анализ и формирование

PDF) → проверка результата (таблица показателей, корреляция и графики в полученном PDF).

8. Тестирование

Корректность математических функций проверяется модульными тестами на xUnit (проект CryptoAnalysis.Tests, запуск dotnet test). Тесты класса Returns проверяют: простую доходность для $[100, 110] = 0,10$; длину ряда $n - 1$; логарифмическую доходность для $[100, 200] = \ln 2$; пустой результат для пустого/одиначного ряда; защиту от неположительных цен. Тесты корреляции проверяют $\rho = 1$ для полной прямой и $\rho = -1$ для полной обратной зависимости. Покрывание сосредоточено на расчётных функциях как наиболее критичных для достоверности результатов.

9. Анализ результатов

По результатам анализа сформирован PDF-отчёт; основные показатели приведены в таблице 3.

Таблица 3 - Статистические показатели BTC и ETH (22.12.2025 - 16.06.2026, 45 свечей)

Показатель	BTC	ETH
Минимальная цена (USD)	63 255,00	1 671,89
Максимальная цена (USD)	97 008,00	3 356,50
Средняя цена (USD)	75 952,27	2 319,08
Медиана (USD)	75 149,00	2 175,06
Ст. отклонение цены (USD)	9 488,99	461,58
Волатильность (σ лог-дох.)	4,41 %	5,66 %
Годовая волатильность	42,13 %	54,08 %

Коэффициент корреляции Пирсона BTC–ETH составил 0,9007 ($N = 44$) - очень сильная положительная линейная связь: активы движутся преимущественно синхронно, и диверсификация только между ними слабо снижает риск. Волатильность ETH (годовая 54,08 %) выше, чем у BTC (42,13 %), то есть Ethereum в рассмотренном периоде был более рискованным активом. Результаты согласуются с известными свойствами рынка (BTC и ETH традиционно сильно коррелированы, ETH волатильнее), что подтверждает корректность алгоритмов на реальных данных.

Эффективность и производительность: система реализует полный цикл одной операцией (POST /reports), воспроизводима и запускается локально одной

командой; при $n \approx 45$ вычисления мгновенны, время отклика определяется сетевыми запросами к CoinGecko, а сложность $O(n \cdot \log n)$ даёт запас при росте объёма данных. Ограничения: гранулярность demo-тарифа (свечи 4 дня), анализ только двух активов и одной пары, базовый набор методов без прогнозирования и выявления аномалий, отсутствие обновления данных по расписанию.

10. Заключение

Спроектирована и реализована система анализа криптовалютного рынка на основе открытых данных. Пройден полный цикл: постановка задачи, проектирование архитектуры, БД и процессов, реализация продукта и алгоритмов, эксперимент и анализ результатов. Система загружает реальные котировки BTC и ETH из CoinGecko, хранит их в PostgreSQL с защитой от дублирования, вычисляет статистики, волатильность и корреляцию Пирсона и формирует PDF-отчёт. На реальных данных получена очень сильная корреляция BTC–ETH (0,9007) и более высокая волатильность ETH, что согласуется с ожидаемым поведением рынка. Направления развития: расширение набора активов и пар, прогнозирование и выявление аномалий, обновление данных по расписанию, развитие средств визуализации.

11. Github

<https://github.com/khongdainam5501git/CryptoAnalysis>